2026학년도 2학기 언어데이터과학

제14강 표준국어대사전 데이터 활용 (1) XML 파일 처리

박수민

서울대학교 인문대학 언어학과

2026년 11월 2일 월요일

지난 시간에 배운 것

1. 발화 표와 화자 표를 `doc_id`와 `speaker_id` 두 열로 이었다.
2. ‘완전’과 ‘아주’ 두 단어를 1만 발화당 수와 ‘완전’의 몫, 두 잣대로 비교했다.
3. 같은 수를 사람마다 다시 세어, 한 사람이 끌고 간 수인지 확인했다.

오늘의 목표

1. 논문(임건수 2025)이 사전 데이터로 무엇을 했는지 말할 수 있다.
2. XML의 요소와 나무 구조를 읽고, JSON과 무엇이 다른지 설명할 수 있다.
3. Python의 `ElementTree`로 경로를 따라 원하는 요소를 꺼낼 수 있다.
4. 표준국어대사전의 파일 88개를 표제어 표 하나로 펼쳐 CSV로 남길 수 있다.

사전 준비 (1) Codespace와 노트북

Codespaces에서 할 일

1. 2강에서 만든 Codespace를 연다.
2. Explorer의 [New File...]를 눌러 저장소 **맨 위**에 `14-xml.ipynb`를 만든다.
3. 오른쪽 위 [Select Kernel] → [Python Environments] → Python 3.12.x

노트북 `14-xml.ipynb`의 첫 셀

```
import xml.etree.ElementTree as ET
from glob import glob
import pandas as pd
from tqdm import tqdm
```

ET는 Python에 들어 있고, `tqdm`은 10강에서 `kiwipiepy`와 함께 설치되었다.

사전 준비 (2) 사전 파일을 Codespaces로

eTL에 올린 파일을 내려받아 Codespace에 올린다

1. eTL에서 `stdict_xml_20261012.zip`(63MB)을 내려받는다.
2. 터미널에서 `mkdir -p data/stdict`로 폴더를 만들고, 탐색기의 그 폴더에 zip을 끌어다 놓는다.

왜 eTL인가

사전은 지금도 고쳐진다. 받는 날이 다르면 표제어 수가 달라진다. 모두가 같은 수를 얻도록 2026년 10월 12일에 받은 **한 버전**을 함께 쓴다. 파일 이름의 `20261012`가 그 날짜다.

사전 준비 (3) 압축 풀기

터미널 입력

```
cd /workspaces/LDS2026/data/stdict
unzip -q stdict_xml_20261012.zip -d xml
ls xml | wc -l
ls xml | head -n 3
```

실행 결과

```
88
1610292_10000.xml
1610292_100000.xml
1610292_105000.xml
```

zip 안에는 폴더가 없어서 `-d xml`로 풀 곳을 정해 주었다. 파일 88개, 모두 657MB다.

1610292 모든 파일에 같다. 내려받을 때 붙은 번호다

10000 이 파일의 마지막 표제어가 가나다순으로 몇 번째인지다. 파일마다 5,000개씩 담았다

그런데 `10000` 다음이 `100000`이다. 이 순서는 §5에서 바로잡는다.

이 파일도 표가 아니다

터미널 입력 — 앞 쪽에 이어서

```
head -n 11 xml/1610292_5000.xml
```

실행 결과 — 이번에는 괄호가 아니라 꺾쇠다

```
<?xml version="1.0" encoding="UTF-8"?>
<channel>
    <title>사전 검색</title>
    <link>https:stdict.korean.go.kr</link>
    <description>사전 검색 결과</description>
    <lastBuildDate>20261012 10:31:03</lastBuildDate>
    <total>436587</total>
    <item>
        <target_code>443</target_code>
        <word_info>
            <word><![CDATA[ㄱ]]></word>
```

1 어미 없는 형용사
논문

2 XML 문법

3 Python으로 읽기

4 표준국어대사전

5 사전 전체를 표로

6 마무리

어간으로 끝나는 말

메신저 대화에서

(4ㄱ) 가: 아 먹을 생각에 설레네… 나: **부럽**

(4ㄴ) 가: 차단거봐 나: **귀엽**

(4ㅅ) 너 내일 학원 **있**??

방송 자막에서

(2ㄷ) 우리는 모두 **똑같**! (놀면 뭐하니 218회) (3ㄱ) **귀찮** (집사부일체 166회)

어간은 '부럽-다', '부러워'처럼 어미와 붙어야 쓰이는데, **어간에서 말을 끝냈다**. 논문은 이것을 '어간의 자립적 사용'이라 부른다. 가장 이른 예는 2007년 기사의 '어쩐지 부럽;'이다.

출처: 임건수(2025)의 (2)·(3)·(4), 185쪽

오늘 읽는 논문

임건수 (2025), 매체 언어에서 형용사 어간의 자립적 사용 양상 연구, 《한국어학》 107, 183–208

방송 9회분의 자막과 국립국어원 메신저 말뭉치(2.0)에서 어미 없이 쓰인 형용사 어간을 찾아 음절 수, 받침, 문장 유형, 쓰는 까닭을 살폈다.

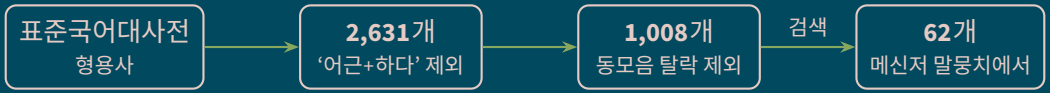
메신저 말뭉치에서 찾은 것 (표 2–4)

- 어미 없이 쓰인 형용사 **62개**. 많이 쓰인 어간은 부럽(77), 무섭(45), 귀찮(39), 귀엽(37), 아쉽(33)
- 2음절 어간이 타입의 56.5%, 어간 끝 받침이 ‘ㅂ’인 것이 타입의 64.9%

그런데 62개를 찾으려면 먼저 **무엇을 찾을지** 알아야 한다.

출처: https://doi.org/10.20405/kl.2025.05.107.183

후보 목록은 사전에서 왔다



동모음 탈락을 뺀 까닭

‘싸다’는 어간 ‘싸’에 ‘-아’가 붙어도 ‘싸’다. 어간만 쓴 것인지 어미가 붙은 것인지 **글자로는 가를 수 없다**.

이번 주의 두 강의

14강 사전의 XML 파일 88개를 **표제어 표 하나**로 펼친다 (오늘)

15강 그 표에서 논문의 후보 목록을 다시 만들어 본다

출처: 임건수(2025) 191쪽

1 어미 없는 형용사

2 **XML 문법**
요소와 나무
문법은 엄격하다

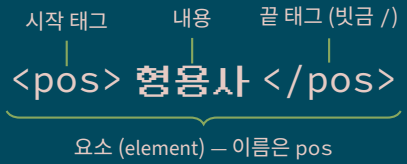
3 Python으로 읽기

4 표준국어대사전

5 사전 전체를 표로

6 마무리

품사 하나를 XML로 적으면



요소(element)

꺾쇠 < >로 적은 **시작 태그**와 **끝 태그** 사이에 내용을 둔 것이다. 두 태그의 이름은 같아야 하고, 끝 태그에는 이름 앞에 /를 붙인다. 내용이 없으면 <pos></pos>를 <pos/>로 줄여 적을 수 있다.

11강의 JSON이라면 {"pos": "형용사"}로 적었을 것이다. 이름이 **값을 감싸는** 것이 XML이다.

요소 안에 요소

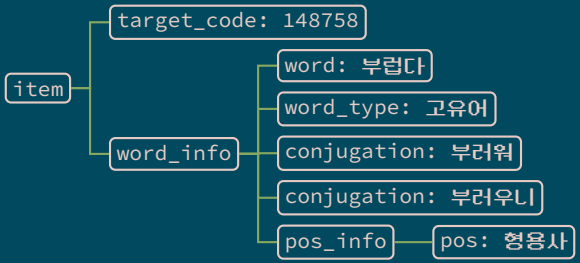
사전의 '부럽다'를 간추린 것 (실제는 §4)

```
<item>
  <target_code>148758</target_code>
  <word_info>
    <word>부럽다</word>
    <word_type>고유어</word_type>
    <conjugation>부러워</conjugation>
    <conjugation>부러우니</conjugation>
    <pos_info>
      <pos>형용사</pos>
    </pos_info>
  </word_info>
</item>
```

`item`	표제어 하나
`target_code`	번호
`word_info`	단어 정보
`conjugation`	활용형
`pos_info`	품사 정보

요소 안에 요소가 들어간다.
안에 들어간 요소를 **자식**, 감싼 요소를 **부모**라고 부른다.

나무로 보기



뿌리는 하나다

가장 바깥의 `item`이 뿌리다. 문서 하나에는 뿌리 요소가 꼭 **하나**만 있다. 뿌리에서 `word_info`, `pos_info`를 거쳐 `pos`에 닿는 길이 Python에서 요소를 꺼내는 **경로**가 된다(§3).

같은 이름을 되풀이하면 목록이다

XML

```
<conjugation>부러
↪  워</conjugation>
<conjugation>부러우
↪  니</conjugation>
```

JSON이라면

```
"conjugation": ["부러워", "부러우니"]
```

XML에는 배열이 없다

목록을 따로 적는 기호가 없다. 같은 이름의 자식을 **차례대로 늘어놓으면** 그것이 목록이다. JSON의 배열처럼 순서에 뜻이 있다. 객체의 이름과 달리 같은 이름이 여러 번 나와도 된다.

그래서 요소 하나를 보고는 그것이 목록의 한 자리인지 알 수 없다. '부러워' 하나뿐인 표제어도 있을 수 있다.

값은 모두 글이다

	JSON (11강)	XML (오늘)
이름과 값	`"pos": "형용사"`	`<pos>형용사</pos>`
값의 꼴	여섯 가지 (문자열, 수, ...)	**글 하나뿐**
수	`"intimacy": 3`	`<target_code>148758</target_code>`
목록	배열 `[ ]`	같은 이름의 요소를 늘어놓는다
이름이 겹치면	안 된다	된다
주석	없다	`<!-- 주석 -->`

148758은 수가 아니다

XML은 내용의 꼴을 적지 않는다. 표제어 번호도 Python에서는 문자열 `'148758'`로 읽힌다. 수로 쓰려면 우리가 `int()`로 바꾸어야 한다. 6강에서 본 대로 문자 `'1'`은 더할 수 없다.

속성 — 태그 안에 적는 이름

사전이 적은 대로

```
<pos_info>
  <pos_code>148758001</pos_code>
  <pos>형용사</pos>
</pos_info>
```

속성으로 적는다면

```
<pos code="148758001">형용사</pos>
```

속성(attribute)

시작 태그 안에 `이름="값"`으로 적는다. 값은 따옴표로 감싼 글이고, 한 요소 안에서 같은 이름은 한 번만 쓴다. 자식 요소와 달리 속성에는 다른 요소를 넣을 수 없다.

어느 쪽으로 적을지는 만드는 사람이 정한다. 표준국어대사전은 속성을 **하나도** 쓰지 않았다. 88개 파일을 모두 세어 확인했다.

글 속의 꺾쇠와 &

꺾쇠는 태그의 시작이라 글에 그대로 쓸 수 없다

`<` < (less than), `>`는 > (greater than)

`&` & (ampersand). &도 이런 이름의 시작이라 그대로 못 쓴다

`{` 코드 포인트 123번 글자, 곧 { (9강)

CDATA — 안쪽을 통째로 글로

```
<word><![CDATA[ㄱ]]></word>
```

<![CDATA[와]]> 사이는 아무것도 풀지 않고 글자 그대로 읽는다. 사전은 표제어와 뜻풀이를 이렇게 감쌌다.

첫 줄의 `encoding="UTF-8"`은 이 파일의 바이트를 UTF-8로 풀라고 **파일이 스스로** 하는 말이다(9강).

문법은 엄격하다

틀린 XML

```
<pos_info>
  <pos>형용사</Pos>
  <pos>R&B
</pos_info>
```

옳은 XML

```
<pos_info>
  <pos>형용사</pos>
  <pos>R&amp;B</pos>
</pos_info>
```

틀린 곳 셋

대소문자 pos와 Pos는 다른 이름이다

끝 태그 연 요소는 반드시 닫고, 연 순서의 반대로 닫는다

& 글 속의 &는 &로 적는다

뿌리가 하나인 것까지, 이 약속을 지킨 문서를 **well-formed**라고 한다.

1 어미 없는 형용사

2 XML 문법

3 **Python으로 읽기**
글을 요소로
요소를 글로

4 표준국어대사전

5 사전 전체를 표로

6 마무리

ET.fromstring — 글을 요소로

노트북 입력 — 앞 쪽의 속성까지 넣어 본다

```
s = '<pos code="148758001">형용사</pos>'
e = ET.fromstring(s)
print(type(s))
print(type(e))
print(e.tag, e.text, e.get('code'))
```

실행 결과

```
<class 'str'>
<class 'xml.etree.ElementTree.Element'>
pos 형용사 148758001
```

.tag 요소의 이름

.text 요소의 내용(글)

.get('code') 속성 code의 값

json.loads가 글을 dict로 바꾸었듯, ET.fromstring은 글을 Element(요소)로 바꾼다.

간추린 표제어를 읽는다

노트북 입력 — §2의 표제어를 줄만 달리 바꾸어 적었다

```
x = '''<item><target_code>148758</target_code>
 <word_info><word>부럽다</word><word_type>고유어</word_type>
  <conjugation>부러워</conjugation><conjugation>부러우니</conjugation>
  <pos_info><pos>형용사</pos></pos_info></word_info></item>'''
item = ET.fromstring(x)
print(item.tag, len(item))
print([child.tag for child in item])
```

실행 결과

```
item 2
['target_code', 'word_info']
```

len은 **자식**의 수다. 손자 요소는 세지 않는다. for로 돌면 자식이 차례로 나온다. 목록처럼 다룰 수 있다.

find와 경로

노트북 입력

```
info = item.find('word_info')
print(info.find('word').text)
print(item.find('word_info/pos_info/pos').text)
print(item.findtext('target_code'))
print(type(item.findtext('target_code')))
```

실행 결과

```
부럽다
형용사
148758
<class 'str'>
```

find 그 이름의 **첫** 자식 요소. 빗금(/)으로 이은 경로를 따라 내려가기도 한다

findtext 찾은 요소의 .text까지 한 번에

경로는 11강의 c['utterance'][1]['form']과 달리 이름으로만 내려간다. 번호는 **문자열**로 나왔다.

findall과 iter

노트북 입력

```
for c in info.findall('conjugation'):
    print(c.text)
print(len(item.findall('conjugation')))
print(len(list(item.iter('conjugation'))))
```

실행 결과

```
부러워
부러우니
0
2
```

findall 그 이름의 자식을 **모두** 목록으로 돌려준다. 같은 이름을 되풀이한 목록을 읽는 함수다

iter 깊이와 상관없이 그 이름의 요소를 **모두** 찾는다. 자기 자신도 본다

`item.findall('conjugation')`이 0인 까닭: `conjugation`은 `item`의 자식이 아니라 손자다. `find`와 `findall`은 자식만, 경로를 주면 그 경로만 본다.

없는 요소

노트북 입력 — 어원(origin)은 없다

```
print(info.find('origin'))
print(info.findtext('origin'))
```

실행 결과

```
None
None
```

일부러 낸 오류 — 따로 실행한다. 아래는 오류 메시지의 마지막 줄

```
info.find('origin').text
```

```
AttributeError: 'NoneType' object has no attribute 'text'
```

find는 못 찾으면 None을 돌려주는데, None에는 .text가 없다. findtext는 멈추지 않고 None을 돌려준다. 없을 수도 있는 요소에는 이쪽을 쓴다.

문법이 틀리면

노트북 입력 — 일부러 틀린 두 줄을 셀 하나에 한 줄씩 실행한다

```
ET.fromstring('<pos>형용사</Pos>')
ET.fromstring('<word>R&B</word>')
```

실행 결과 — 오류 메시지의 마지막 줄

```
ParseError: mismatched tag: line 1, column 10
ParseError: not well-formed (invalid token): line 1, column 9
```

칸은 0부터 센다. 첫 줄은 끝 태그의 이름이 짝과 맞지 않는 P(10번째 칸)에서 멈췄다. 둘째 줄은 &B 뒤에 ;가 와야 하는데 <를 만난 9번째 칸에서 멈췄다. &는 7번째 칸이다. 메시지는 틀린 곳이 아니라 **알아챈 곳**을 알려 준다.

ET.tostring — 요소를 다시 글로

노트북 입력

```
d = ET.Element('example')
d.text = '<부럽다> & <귀엽다>'
print(ET.tostring(d, encoding='unicode'))
print(ET.fromstring(ET.tostring(d, encoding='unicode')).text)
```

실행 결과

```
<example>&lt;부럽다&gt; &amp; &lt;귀엽다&gt;</example>
<부럽다> & <귀엽다>
```

ET.Element로 요소를 새로 만들고 내용을 넣었다. 글로 적을 때 꺾쇠와 &는 저절로 <, &가 되고, 다시 읽으면 원래 글로 돌아온다. encoding='unicode'는 바이트가 아니라 str로 달라는 뜻이다. XML을 손으로 이어 붙여 만들지 말고 이렇게 만드는 까닭이다.

JSON과 XML, 두 모듈

하는 일	`json` (11강)	`xml.etree.ElementTree` (오늘)
글 읽기	`json.loads(s)` → `dict`	`ET.fromstring(s)` → `Element`
파일 읽기	`json.load(f)`	`ET.parse(path).getroot()`
글로 적기	`json.dumps(v)`	`ET.tostring(e, encoding=…)`
이름으로	`d['word_info']`	`e.find('word_info')`
없으면	`KeyError`	`None`
경로	`d['word_info']['pos']`	`e.find('word_info/pos')`
목록	`d['conjugation']`	`e.findall('conjugation')`
내용	값 그 자체	`e.text` (언제나 글)

`lxml`이라는 외부 패키지도 같은 이름의 함수(`fromstring`, `find`, `findall`, `iter`)를 쓴다. 더 빠르고 기능이 많지만 설치가 필요하다. 오늘은 들어 있는 것을 쓴다.

확인 질문

여기까지 정리

1. JSON의 `"conjugation": ["부러워", "부러우니"]`를 XML로는 어떻게 적는가? XML에는 무엇이 없는가?
2. `<target_code>148758</target_code>`를 읽어 1을 더하려면 무엇을 해야 하는가?
3. `item.findall('conjugation')`이 0을 돌려준 까닭은? 두 개를 다 얻는 방법을 둘 들어 보자.
4. `find(…).text`와 `findtext(…)`는 요소가 없을 때 어떻게 다른가?
5. 뜻풀이에 'A<B'를 그대로 적으려면 어떻게 해야 하는가? 방법을 둘 들어 보자.

1 어미 없는 형용사

2 XML 문법

3 Python으로 읽기

4 **표준국어대사전**
사전 데이터
파일 하나

5 사전 전체를 표로

6 마무리

국립국어원 표준국어대사전

오늘 쓰는 버전

- 국립국어원이 1999년에 처음 펴냈고, 지금은 누리집에서 고쳐 나간다.
  - `https://stdict.korean.go.kr`
- 2026년 10월 12일에 받은 버전의 표제어는 **436,587개**.

이용 조건 — 누리집의 저작권 정책

크리에이티브 커먼즈 저작자표시-동일조건변경허락 2.0 대한민국. 출처를 밝히면 고쳐 쓰고 나누어 줄 수 있고, 그 결과물도 같은 조건으로 나눈다. 다만 **출전이 있는 용례**(소설 등에서 따온 문장)는 원저작권자의 허락이 필요하다.

오늘 버전을 eTL로 나누어 줄 수 있는 것도 이 조건 덕분이다. 그래도 `data/` 아래에 두고 커밋하지 않는다.

파일 하나 열기

노트북 입력 — '부럽다'가 든 파일

```
path = 'data/stdict/xml/1610292_165000.xml'
root = ET.parse(path).getroot()
print(root.tag)
print([child.tag for child in root][:7])
print(root.findtext('total'), len(root.findall('item')))
```

실행 결과

```
channel
['title', 'link', 'description', 'lastBuildDate', 'total', 'item', 'item']
436587 5000
```

ET.parse가 파일을 읽고 getroot()가 뿌리를 준다. 뿌리 channel 아래에 설명 다섯과 item 5,000개가 나란히 있다. total은 사전 전체의 표제어 수다.

‘부럽다’를 찾는다

노트북 입력

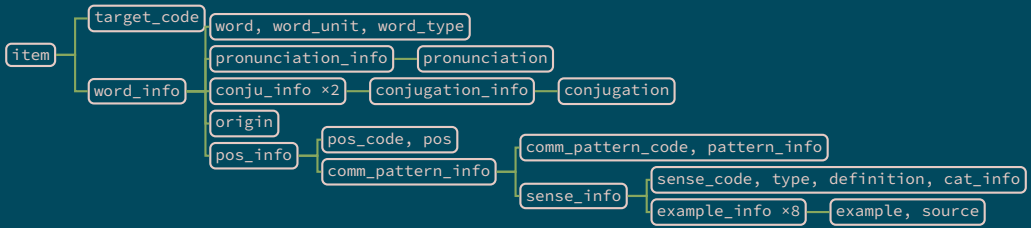
```
found = [it for it in root.findall('item')
         if it.findtext('word_info/word') == '부럽다']
print(len(found))
item = found[0]
print(item.findtext('target_code'))
print([child.tag for child in item.find('word_info')])
```

실행 결과

```
1
148758
['word', 'word_unit', 'word_type', 'pronunciation_info', 'conju_info',
↪  'conju_info', 'origin', 'pos_info']
```

이제 item은 **진짜** ‘부럽다’다.

사전의 나무



§2의 표제어는 이 나무의 간추린 꼴이었다

활용형은 conju_info, conjugation_info, conjugation 세 겹 안에 있고, 뜻풀이는 다섯 겹 아래에 있다. 품사가 둘이면 pos_info가 둘, 뜻이 여럿이면 sense_info가 여럿이다.

들여쓰기도 글이다

노트북 입력

```
print(repr(item.text))
print(repr(item.findtext('word_info/word')))
```

실행 결과

```
'\n\t\t'
'부럽다'
```

repr 보이지 않는 글자까지 이스케이프로 보여 준다. \n은 줄바꿈, \t는 탭이다

item의 내용은 <item> 바로 뒤의 줄바꿈과 탭 둘이다. JSON과 달리 XML에서는 태그 사이의 공백도 **글**로 남는다. 그래서 내용이 아니라 자식을 담은 요소의 .text는 쓰지 않는다. 한편 word의 <![CDATA[…]]>는 사라지고 글만 남았다.

활용형과 뜻풀이

노트북 입력

```
info = item.find('word_info')
print([c.text for c in info.iter('conjugation')])
print(info.findtext('pos_info/comm_pattern_info/sense_info/definition'))
print(len(list(info.iter('example'))))
```

실행 결과

```
['부러워', '부러우니']
남의 좋은 일이나 물건을 보고 자기도 그런 일을 이루거나 그런 물건을 가졌으면 하고 바라는 마음이 있다.
8
```

세 겹 아래의 활용형은 iter가 깊이와 상관없이 모아 온다. 뜻풀이는 경로를 끝까지 적었다. 용례는 8개이고, 그 가운데 셋은 소설에서 따온 것이라 source(출전)가 붙어 있다. 앞 쪽의 이용 조건이 말한 용례다.

CDATA 안의 {

노트북 입력 — 다른 파일의 '괄호'

```
root2 = ET.parse('data/stdict/xml/1610292_40000.xml').getroot()
for it in root2.findall('item'):
    if it.findtext('word_info/word') == '괄호':
        print(it.findtext('.//definition'))
```

실행 결과

```
문장 부호의 하나. 소괄호(( )), 중괄호(&#123; &#125;), 대괄호([ ])가 있다.
```

`.//` 경로 앞에 붙이면 '어느 깊이든'이다. 처음 만나는 definition을 찾는다

중괄호 자리에 { }가 남았다. 사전이 { }를 글자 참조로 바꾼 뒤 CDATA로 감싸서, CDATA 안이라 **풀리지 않았다**. 이런 뜻풀이가 152개다.

1 어미 없는 형용사

2 XML 문법

3 Python으로 읽기

4 표준국어대사전

5 **사전 전체를 표로**
- 표제어 표
- 저장하기
- 저장소에 남기기

6 마무리

파일 이름 모으기

노트북 입력

```
paths = sorted(glob('data/stdict/xml/*.xml'))
print(len(paths))
for p in paths[:3]:
    print(p)
```

실행 결과

```
88
data/stdict/xml/1610292_10000.xml
data/stdict/xml/1610292_100000.xml
data/stdict/xml/1610292_105000.xml
```

글자의 순서는 수의 순서가 아니다

sorted는 문자열을 **글자 하나씩** 비교한다. 10000.xml과 100000.xml은 여섯째 글자에서 .(46)이 0(48)보다 앞선다. 5000은 한참 뒤로 갔다.

수의 순서로 줄 세우기

노트북 입력

```
def number(path):
    return int(path.split('_')[-1].removesuffix('.xml'))

paths = sorted(paths, key=number)
for p in paths[:3]:
    print(p)
```

실행 결과

```
data/stdict/xml/1610292_5000.xml
data/stdict/xml/1610292_10000.xml
data/stdict/xml/1610292_15000.xml
```

key=number는 “이름 대신 number(이름)을 비교하라”는 뜻이다. 이제 사전의 순서대로 나온다.

'1610292_10000.xml' → '10000.xml' → '10000' → 10000

파일 88개를 차례로

노트북 입력 — 막대가 끝까지 찰 때까지 기다린다

```
rows = []
for path in tqdm(paths):
    for item in ET.parse(path).getroot().findall('item'):
        code = int(item.findtext('target_code'))
        info = item.find('word_info')
        for pos in info.findall('pos_info/pos'):
            rows.append({'target_code': code,
                         'word': info.findtext('word'),
                         'word_unit': info.findtext('word_unit'),
                         'word_type': info.findtext('word_type'),
                         'pos': pos.text})
```

파일마다, 표제어마다, **품사마다** `dict` 하나를 담는다. 번호는 `int()`로 수로 바꾸었다.

진행 막대 tqdm

실행 결과 — 막대가 끝까지 차면 끝난 것이다

```
100%|██████████| 88/88 [00:14<00:00,  6.22it/s]
```

`tqdm(paths)`는 `paths`를 그대로 하나씩 내준다. 반복은 그대로이고 막대만 더해진다.

읽는 법

`88/88`	지금까지 읽은 파일 수와 전체 파일 수
`[00:14<00:00]`	걸린 시간과 남은 시간(분:초). 컴퓨터마다 다르다
`6.22it/s`	1초에 도는 횟수. 여기서는 1초에 파일 6개쯤

오래 걸리는 반복에는 `tqdm`을 씌워 두면, 멈춘 것인지 아직 도는 것인지 알 수 있다.

표제어 표

노트북 입력

```
headwords = pd.DataFrame(rows)
print(headwords.shape)
print(headwords.head(3))
```

word_unit 단어, 구, 속담, 관용구

word_type 고유어, 한자어, 외래어, 혼종어

실행 결과

```
(439566, 5)
   target_code     word word_unit word_type pos
0          443        ㄱ       단어       고유어  명사
1          445   ㄱㄴㄷ-순       단어       혼종어  명사
2       386697  ㄱㄴㄷ-자례       단어       혼종어  명사
```

표제어는 436,587개인데 행은 439,566개다. 표제어의 -는 사전의 **붙임표**로, 구성 요소의 경계를 보인다.

표제어보다 행이 많다

노트북 입력

```
print(headwords['target_code'].nunique())
print(headwords[headwords['word'] == '바삭-하다'])
```

실행 결과

```
436587
        target_code    word word_unit word_type  pos
139773       428825  바삭-하다       단어       고유어   동사
139774       428825  바삭-하다       단어       고유어  형용사
```

한 행은 '표제어 하나의 품사 하나'다

'바삭하다'는 형용사이자 동사(바삭 소리가 나다)라서 두 행이다. 이렇게 2,979행이 늘었다.

비어 있는 칸

노트북 입력

```
print(headwords.isna().sum().to_dict())
na = headwords['word_type'].isna()
print(headwords.loc[na, 'word_unit'].value_counts().to_dict())
```

실행 결과

```
{'target_code': 0, 'word': 0, 'word_unit': 0, 'word_type': 11322, 'pos': 0}
{'속담': 7436, '관용구': 3886}
```

속담과 관용구에는 어종이 없다

‘가는 날이 생일’ 같은 속담과 관용구에는 <word_type>이 없다. findtext의 None이 표에서 NaN이 되었다. find(…).text였다면 첫 속담에서 멈췄을 것이다.

품사별로 세어 보면

노트북 입력

```
counts = headwords['pos'].value_counts()
print(counts.head(6).to_string())
adj = headwords[headwords['pos'] == '형용사']
print(len(adj))
print(adj['word'].str.endswith('-하다').sum())
```

논문의 첫 단계를 흉내 내면

형용사 13,400행에서 '-하다'로 끝나는 7,747행을 빼면 5,653행. 논문의 2,631개의 **두 배가 넘는다**. 까닭은 15강에서 찾는다.

실행 결과

```
pos
명사        270046
품사 없음     70774
동사         56474
형용사        13400
부사         12325
구           11322
13400
7747
```

to_string()은 맨 아래의 Name, dtype 줄을 빼고 찍는다.

CSV로 저장하고 다시 읽어 본다

노트북 입력

```
headwords.to_csv('data/stdict/headwords.csv', index=False)
back = pd.read_csv('data/stdict/headwords.csv')
print(back.shape)
print(back.isna().sum().sum())
print(back['target_code'].dtype)
```

실행 결과

```
(439566, 5)
11322
int64
```

그대로 돌아왔다

빈칸은 어종이 없는 11,322칸 그대로이고, 번호는 수로 돌아왔다. 파일은 19MB다. 657MB의 XML에서 다섯 열만 남겼다.

15강은 XML이 아니라 이 CSV에서 시작한다.

확인 질문

사전을 표로 펼치며

1. `sorted`만 했을 때 `1610292_5000.xml`이 맨 앞에 오지 않은 까닭은?
2. 표제어는 436,587개인데 표는 439,566행이다. 한 행은 무엇인가?
3. `word_type`을 `findtext`가 아니라 `find(…).text`로 꺼냈다면 어떻게 되었을까?
4. '괄호'의 뜻풀이에 `{`가 남은 까닭을 CDATA로 설명해 보자.
5. 형용사 가운데 '-하다'로 끝나는 것을 뺀 5,653이 논문의 2,631보다 훨씬 많다. 짐작 가는 까닭은?

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026
git status
```

실행 결과 (사전과 CSV는 data/ 아래라 3강의 .gitignore에 걸린다)

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        14-xml.ipynb

nothing added to commit but untracked files present (use "git add" to track)
```

커밋하기

터미널 입력

```
git add 14-xml.ipynb
git commit -m "14강 실습 노트북: 표준국어대사전을 표로"
```

실행 결과

```
[main a1b2c3d] 14강 실습 노트북: 표준국어대사전을 표로
 1 file changed, 664 insertions(+)
 create mode 100644 14-xml.ipynb
```

664줄이 모두 노트북이다. 사전(zip 63MB, XML 657MB)과 CSV(19MB)는 올라가지 않는다.

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 4, done.
Writing objects: 100% (3/3), 3.65 KiB | 3.65 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

버전과 방법을 남긴다

노트북에는 버전(20261012)과 무엇을 한 행으로 쳤는지가 코드로 남았다. 같은 zip을 받은 사람은 누구나 [Run All]로 행의 순서까지 같은 CSV를 얻는다.

1 어미 없는 형용사

2 XML 문법

3 Python으로 읽기

4 표준국어대사전

5 사전 전체를 표로

6 **마무리**
요약
다음 시간

오늘 쓴 것

하는 일	오늘의 코드
XML 글을 요소로	`ET.fromstring(s)`
XML 파일을 요소로	`ET.parse(path).getroot()`
요소를 XML 글로	`ET.tostring(e, encoding='unicode')`
이름·내용·속성	`e.tag`, `e.text`, `e.get('code')`
경로로 첫 요소	`e.find('word_info/pos_info/pos')`
내용까지 한 번에	`e.findtext('word_type')` (없으면 None)
같은 이름의 자식 모두	`e.findall('pos_info/pos')`
깊이와 상관없이 모두	`e.iter('example')`, 경로 앞의 `.//`
수의 순서로 정렬	`sorted(paths, key=number)`
표제어 세기	`headwords['target_code'].nunique()`

새로 배운 것은 `xml.etree.ElementTree`, `sorted`의 `key`, `tqdm`이다. 표를 다루는 도구는 11강의 것 그대로다.

오늘 배운 것

XML

- 이름이 값을 감싼다. 뿌리는 하나
- 목록은 같은 이름을 늘어놓는다
- 값은 모두 글, 태그 사이 공백도

ElementTree

- `find`·`findall`은 자식만
- `iter`와 `.//`는 깊이 상관없이
- 없을 수 있으면 `findtext`

표준국어대사전

- 2026-10-12 버전, 표제어 436,587개
- CC BY-SA 2.0 KR, 용례는 따로
- CDATA 안의 참조는 안 풀린다

표제어 표

- 파일은 수의 순서로 읽는다
- 한 행은 표제어 하나의 품사 하나
- 속담·관용구에는 어종이 없다

확인 질문

오늘 것을 정리해 봅시다

1. 같은 표제어를 JSON과 XML로 적을 때 다른 점을 셋 들어 보자.
2. `<pos code="148758001">형용사</pos>`에서 `.tag`, `.text`, `.get('code')`는 각각 무엇인가?
3. '부럽다'의 활용형 둘을 꺼내는 코드를 `iter`로 한 번, 경로와 `findall`로 한 번 적어 보자.
4. 사전의 버전(받은 날)을 적어 두어야 하는 까닭을 오늘 본 두 수로 설명해 보자.
5. 15강에서 '부럽다'의 어간 '부럽'을 얻으려면 표제어 표의 어느 열을 어떻게 다루어야 할까?

다음 시간에 배울 것 (11월 4일 수요일)

제15강 표준국어대사전 데이터 활용 (2) 표제어 추출 및 분석

오늘 만든 `headwords.csv`에서 시작한다. 논문의 후보 목록(형용사 2,631개, 1,008개)을 다시 만들어 보고, 오늘의 5,653이 2,631과 왜 다른지 살핀다. 표제어 **창연-하다**02가 첫 실마리다.

미리 생각해 둘 것

논문은 어미 없이 쓰인 어간의 65%가 'ㅂ' 받침이라고 했다. 그런데 사전의 형용사에서 'ㅂ' 받침 어간은 **원래** 몇 %일까? 그 수를 모르면 65%가 많은지 알 수 없다.

CSV는 Codespace 안에만 있다

`headwords.csv`는 `data/` 아래라 GitHub에 없다. Codespace를 새로 만들었다면 zip을 다시 올리고 오늘 노트북을 [Run All]로 돌려 만든다.